NS-P02: Complete Project & Viva Guide

Manufacturing Security Policy Simulation with Read-Only Reporting

GitHub: https://github.com/JaiKukreja-7/firewall

Hosted project: https://ns-p02-firewall-lab.onrender.com

Prepared 5 October 2026

Contents

1. Project identity and access
2. One-minute explanation
3. Business problem and threat model
4. Scope: what runs and what is conceptual
5. Security zones, addresses and topology
6. Technology stack and why each part was chosen
7. Core security policy
8. How a simulated flow is processed
9. NAT and PAT explained with an example
10. Reply matching versus a stateful firewall
11. Real read-only database demonstration
12. User interface and classroom controls
13. Backend API reference
14. Source-code map and runtime dependencies
15. Test results and what they prove
16. Mapping to the original Word report
17. Hosting and deployment lifecycle
18. Local reproduction and development
19. Five-to-ten-minute classroom demonstration
20. Common teacher questions and accurate answers
21. Troubleshooting during presentation
22. Limitations and next engineering steps
23. Key terminology
24. References and final presentation statement

1. Project identity and access

Project NS-P02: Manufacturing Network Security Policy Simulation with Read-Only Reporting. The project is based on the Firewall and NAT Security Lab manufacturing scenario in the supplied Word report. This guide describes the implementation actually delivered, rather than repeating unverified results from that report.

Item	Value
Project code	NS-P02
Domain	Network security; IT/OT segregation; firewall policy; NAT/PAT; read-only reporting
GitHub repository	https://github.com/JaiKukreja-7/firewall
Hosted project URL	https://ns-p02-firewall-lab.onrender.com
Reporting page	https://ns-p02-firewall-lab.onrender.com/reporting.html
Health endpoint	https://ns-p02-firewall-lab.onrender.com/health
Guide date	5 October 2026

The hosted URL was supplied by the project owner. Public-site checks from the preparation workspace were blocked by its outbound network policy. The application was tested in local live-server and static-hosting modes; do not present those checks as independent verification of the current public deployment. Check the site in your own browser before class.

2. One-minute explanation

This project demonstrates how a manufacturing company can reduce unauthorized access by dividing its network into four security zones: Internet, Corporate IT, Factory OT and Administration. A default-deny policy permits only selected traffic. Corporate web and approved DNS requests are allowed, but ordinary Corporate-to-OT access is denied. The administrative jump host receives narrow management permissions. PAT illustrates how allowed outbound connections share one public address. A separate live reporting service uses a real SQLite database to demonstrate reading production telemetry while rejecting writes.

The browser models network-policy decisions; it is not a packet-forwarding firewall or Cisco IOS emulator. The SQLite read-only controls are actual database behavior. These two kinds of evidence must be described separately.

3. Business problem and threat model

Threat	Manufacturing impact	Control demonstrated
Flat IT/OT network	A compromised employee workstation can attempt lateral access to factory systems.	Separate address zones and Corporate-to-OT denial.
Cleartext Telnet	Credentials may be intercepted on network paths.	Routed TCP 23 denial and SSH-only Cisco VTY configuration.
Uncontrolled egress	Unauthorized services can enable exfiltration or command-and-control.	HTTPS/approved DNS permits followed by default deny.
Unauthorized SCADA management	Non-admin users may attempt access to monitoring/control endpoints.	Dedicated jump-host source and restricted destination/service checks.
Writable reporting connection	A reporting user could accidentally or deliberately modify production data.	SQLite read-only URI, query-only setting and restricted authorizer.

These controls reduce selected access paths. They do not prove that ransomware, exfiltration, exploitation or physical compromise is prevented. Allowed HTTPS can carry malicious content; an IP-based policy is not a substitute for user authentication.

4. Scope: what runs and what is conceptual

Component	Actual implementation	Boundary
Network dashboard	HTML/CSS/JavaScript executing in the browser; editable traffic descriptions, decisions, PAT mappings and logs.	Does not transmit the modeled packets, configure VLANs or establish TLS/SSH/RDP sessions.
Python policy model	model.py evaluates equivalent representative policies; tested independently.	The dashboard maintains its own browser state rather than calling this model for every flow.
Read-only reporting	Python server executes real SQLite queries on synthetic telemetry.	Database is local to the application, not across an industrial VLAN.
Reporting source/destination policy	Separate reporting_policy function permits one exact host/service tuple.	Not enabled in the baseline dashboard and not an actual network-access control protecting the public API.
Cisco build kit	Router/switch/ISP configuration text and assembly guide.	Not executed in Packet Tracer or IOS; no genuine .pkt artifact.
Public hosting	Render configuration supports a Python web service; Vercel configuration supports static assets.	A public URL is not evidence of real network segmentation.

5. Security zones, addresses and topology

Zone	Subnet / VLAN	Gateway / key endpoints	Role
Internet / WAN	203.0.113.0/30 WAN	ISP 203.0.113.1; security gateway 203.0.113.2	Untrusted external zone.
External lab services	198.51.100.0/24 and simulated DNS	Web 198.51.100.10; resolver 8.8.8.8	External service destinations in the scenario.
Corporate IT	192.168.10.0/24; VLAN 10	Gateway .1; PC .50; CAD PC .51	Employee and engineering workstations.
Factory OT	192.168.20.0/24; VLAN 20	Gateway .1; SCADA .10; PLC stand-in .50	Critical operational systems.
Administration	192.168.30.0/24; VLAN 30	Gateway .1; jump host .10; console .11	Restricted management.
Reporting supplement	Corporate host 192.168.10.60	Proposed OT API 192.168.20.10 TCP 8443	New explicit design assumption for a separate reporting demo.

```
Conceptual network topology (not a running Cisco topology):
External web / DNS -> ISP router -> Security gateway -> 802.1Q trunk -> Switch
|-- VLAN 10: Corporate
|-- VLAN 20: Factory OT
`-- VLAN 30: Administration

Actual hosted software path:
Browser -> Render HTTPS endpoint -> Python server.py -> local SQLite database
|
`-> JavaScript policy model -> browser-local sessions, PAT table and log
```

The /24 internal networks use mask 255.255.255.0; WAN /30 uses 255.255.255.252. The address 8.8.8.8 is a scenario destination; the project does not configure or own the actual Google DNS service. Documentation IP ranges are example network addresses, not the public Render server addresses. Same-VLAN traffic is modeled as bypassing the routed gateway.

6. Technology stack and why each part was chosen

Technology	Use	Reason / requirement
HTML5	Dashboard, reporting UI and documentation pages.	Runs in a standard browser.
CSS3	Responsive cards, tables, colors, focus states and print layouts.	Readable on desktop/mobile; no UI framework required.
Vanilla JavaScript	Policy evaluation, state, validation, logs, reply matching and JSON export.	No frontend package installation; offline dashboard capability.
Python 3.10+	HTTP server, independent policy model, site build and tests.	Runtime uses standard-library modules only.
http.server / urllib / mimetypes	Serve static assets, JSON endpoints, MIME types and path handling.	Small educational service; not a production-grade web server.
sqlite3 / SQLite	Synthetic telemetry and real read-only database controls.	Included with the supplied Python runtime; no separate database server.
unittest / ipaddress	Automated Python checks and IP/subnet policy processing.	Standard-library test and address tools.
Playwright + existing Chromium	Development-time browser automation.	Used in the cloud to verify UI behavior; not required on the teacher's computer or Render runtime.
Git / GitHub	Source history, repository sharing and hosting integration.	Deployment source and reproducibility.
Render	Static dashboard plus live Python reporting API.	Runs server-side SQLite checks without software installation on the student's Mac.
Vercel	Alternative static deployment.	Dashboard and recorded reporting evidence; no live SQLite backend in this configuration.
python-docx / ReportLab / python-pptx	Generate Word/PDF reports and a slide deck.	Document-generation dependencies in the preparation environment only; not web-runtime dependencies.
Cisco IOS configuration text	Optional Packet Tracer build kit.	Reference implementation; Cisco runtime validation remains unrun.

The runtime does not use React, Flask, Django, Node.js, PostgreSQL, Docker or a commercial firewall. A TCP 8443 policy example is not a deployed PostgreSQL or HTTPS OT service. No API keys or production credentials are required for the educational deployment.

7. Core security policy

Initiator	Destination	Service	Decision / reason
Corporate or Admin	External Internet	TCP 443	Permit outbound HTTPS after internal-zone restrictions.
Corporate or Admin	8.8.8.8 only	UDP 53	Permit approved DNS; deny alternative resolver destinations.
Corporate or Admin	198.51.100.10 only	ICMP echo	Permit a documented lab diagnostic exception.

Initiator	Destination	Service	Decision / reason
192.168.30.10 jump host	Internal Corporate / OT systems	TCP 22	Permit restricted SSH management.
192.168.30.10 jump host	SCADA 192.168.20.10	TCP 3389	Permit only this SCADA management target.
192.168.30.10 jump host	Gateway addresses	TCP 22	Permit gateway management in the model.
Ordinary Corporate host	Factory OT / Admin	Any initiated cross-zone traffic	Deny lateral access.
Factory OT	Other zones / Internet	Initiated cross-zone traffic	Deny; limited replies handled separately.
Internet	New connections to gateway/internal targets	Unsolicited traffic	Deny.
Any routed cross-zone source	Another zone	TCP 23	Deny Telnet.
Same internal zone	Same internal zone	Direct local traffic	Gateway bypass is modeled; gateway ACLs cannot police it.
Any otherwise unmatched cross-zone flow	Any	Any	Default deny.

Policy rules are ordered. Restrict internal destinations before the broad outward TCP 443 permit so that HTTPS is not an accidental path into OT. The gateway destination check restricts management independently. IP addresses and port numbers are user-entered simulation inputs, not authenticated identities.

8. How a simulated flow is processed

• Validate IPv4 address fields, protocol and port ranges. TCP, UDP and ICMP are the supported protocol labels.

• Classify source and destination into Corporate, Factory OT, Admin, Internet or an unrecognized address category.

• Apply ordered policy branches and produce PERMIT or DENY with the matched reason.

• For a permitted cross-zone flow, record the client/server tuple. If its destination is Internet, allocate or reuse a simulated PAT port.

• Append the flow decision to a log and update UI counters and the translation table.

• For a simulated reply, compare the reverse tuple with the session table; allow a matching tuple and reject an unmatched one.

ICMP is represented using a placeholder destination-port field and a source-port/identifier field. This is an educational simplification, not a complete ICMP packet format. Flow records are not captured packets. Browser state clears on reload or Reset; it is independent of the Python API state. Public users do not share browser-local dashboard sessions.

9. NAT and PAT explained with an example

NAT changes an address in a translated flow. PAT adds port differentiation so several private hosts can share one public address. The model allocates mappings starting at public port 10001 and reuses a mapping for a repeated identical flow. Different workstations using the same source port receive different public ports.

Original client	Remote service	Illustrative translated client
192.168.10.50:51234	198.51.100.10:443	203.0.113.2:10001
192.168.10.51:51234	198.51.100.10:443	203.0.113.2:10002

Inside local means the private client socket; inside global means its translated public socket; outside means the remote endpoint. Factory OT has no PAT in the design. Its outbound restriction is enforced by policy, not merely by absence of a translation rule. PAT demonstrates source-address multiplexing; it does not encrypt data, hide every topology clue or replace firewall policy.

10. Reply matching versus a stateful firewall

The browser and Python models remember permitted flow tuples. Reverse traffic must match the server address, protocol and ports; PAT replies target the public translated port and map back to the original client. The interface can show a matching reply permitted and a wrong-source reply denied.

This is simplified session matching, not full TCP stateful inspection. The model does not validate SYN/SYN-ACK/ACK handshakes or sequence numbers, implement FIN/RST teardown, or expire idle sessions. The Cisco established ACL keyword checks ACK/RST flags, not whether a real session exists. The static ACL tests intentionally demonstrate that an ACK-shaped packet can match an ACL permit without prior state. Do not claim SYN-flood protection or full SPI.

11. Real read-only database demonstration

At service startup, Python creates a temporary SQLite fixture containing a telemetry table. Each row has machine, status and units fields. The synthetic data are PLC-01 / RUNNING / 1240 and PLC-02 / RUNNING / 980. These are sample records, not measurements from a factory.

```
SELECT machine, status, units
FROM telemetry
ORDER BY machine;
```

• The query connection opens the existing database with SQLite URI mode=ro.

• PRAGMA query_only=ON provides an additional read-only setting.

• A SQLite authorizer allows only SELECT and reads from the telemetry table; other operation categories are denied.

• Tests attempt UPDATE, DELETE, INSERT, DROP TABLE, query_only override and ATTACH DATABASE; all six must fail.

• A separate verification query confirms that the original data remain unchanged.

The network exception is evaluated separately: exactly 192.168.10.60 -> 192.168.20.10, TCP 8443 is permitted; another source, another destination, another protocol or another port is denied. This tuple is a proposed reporting design, not the actual listener address of the hosted application. The baseline network dashboard and Cisco configuration continue to deny Corporate-to-OT initiations; they do not silently enable this exception.

The public reporting API is intentionally accessible for an educational demonstration and returns fixed synthetic telemetry. It does not authorize requests based on the modeled source IP and does not accept user-supplied SQL. Local file owners or administrators could open another writable connection. These controls protect the demonstration query connection; they do not make the database file permanently immutable or implement industrial database replication.

12. User interface and classroom controls

Control / page	What to demonstrate
Live lab	Pick a preset or enter source/destination/protocol/ports; show decision reason and PAT result.
Counters	Show evaluated, permitted, denied and mapped flows.
Return-traffic inspection	Test matching and wrong-source replies after an allowed cross-zone flow.
Validation & evidence	Run 20 browser checks, inspect expected/observed outcomes and export JSON.
PAT table	Show two client addresses sharing one public address with distinct ports.
Decision log filter	Switch between all, permitted and denied flows.
Report coverage	Explain which requirements are modeled and which are not runtime-validated.

Control / page	What to demonstrate
Read-only reporting page	Refresh the 13 reporting checks and show synthetic telemetry.
Report & presentation links	Open the generated submission report, PDF and PowerPoint.
Print / Save as PDF	Save reporting evidence or the locally generated final project page.

The exported browser JSON contains generation time, scenarios/checks, events and translations. It records model decisions and does not represent a packet capture or database audit from an industrial site. Failed checks must be investigated rather than hidden.

13. Backend API reference

Method / route	Purpose	State / response
GET /	Serve dashboard.	Static page; browser executes its own policy model.
GET /health	Hosting readiness endpoint.	status=ok and educational scope.
GET /api/report	Execute read-only SQLite telemetry query.	Two synthetic rows; live-local-sqlite mode.
GET /api/reporting-validation	Execute reporting validation.	13 results, scope, policy tuple and telemetry.
GET /api/state	Inspect Python policy API state.	events and translations; separate from browser state.
POST /api/flow	Evaluate Python-model flow.	source, destination, protocol, port; optional source_port.
POST /api/reply	Evaluate Python-model reply.	source, destination, protocol, source_port, destination_port.
POST /api/demo	Run 16 Python flow scenarios.	Resets model to demo state and returns results.
POST /api/reset	Reset Python API model.	status=reset.
GET /reporting.html	Serve reporting interface.	Chooses live backend results or recorded static evidence.
GET /submission.html	Serve report and presentation-script page.	Generated documentation.
GET /teacher-guide.html	Serve this comprehensive guide.	Printable HTML; PDF/DOCX links.

POST requests have a 4,096-byte body limit and must contain a JSON object. Static file requests are resolved inside the site root, with parent-directory escape rejected. These checks were exercised during local hosting verification. The server is an educational Python HTTPServer, not a hardened multi-user production backend; Python API model state is shared within that process, whereas normal dashboard state remains per browser.

14. Source-code map and runtime dependencies

File / directory	Responsibility
web/index.html	Browser UI, JavaScript policy evaluator, sessions, PAT, logs, 16 presets and 20-check suite.
web/reporting.html	Reporting UI; fetch live validation, or display recorded build evidence when static.
model.py	Independent Python policy model, PAT/reply matching and scenario runner.

File / directory	Responsibility
reporting_demo.py	Database fixture, read-only connection, reporting tuple and 13 checks.
server.py	HTTP/JSON service, local synthetic SQLite startup and static assets.
tools/build_site.py	Standard-library static asset build and reporting-evidence generation.
tools/build_teacher_guide.py	One content source for this guide in HTML, Markdown, Word and PDF.
tests/test_model.py	Six Python model test methods.
tests/test_cisco_policy.py	Seven methods for the limited ACL evaluator and boundary matrices.
tests/test_reporting.py	Three methods for read-only reporting, read-only URI enforcement and fixture preservation.
tests/browser.cjs	Playwright UI tests using existing cloud browser tools.
tests/deployment.cjs	Live versus static reporting, navigation/downloads, mobile, health and API checks.
configs/	Cisco gateway, switch and ISP text configurations.
docs/	Packet Tracer assembly, requirement audit, reporting design and deployment explanations.
deliverables/	Reports, slide deck, evidence, local no-download helper scripts and generated guides.
site/	Prebuilt public assets used by static hosting and the Python service.
render.yaml / vercel.json	Platform deployment settings.

Runtime has no third-party Python dependency installation step. Document generators and Playwright are development tools already available in the preparation environment. The SQLite fixture resets on a restart, which is intentional for synthetic demonstrations. No production secrets or external database credentials are embedded in the project.

15. Test results and what they prove

Suite	Verified result	Meaning
Browser validation	20 / 20 checks passed in preparation environment.	16 policy scenarios + PAT uniqueness + HTTPS reply + wrong-source denial + OT RDP reply.
Python unittest	16 / 16 methods passed.	6 model + 7 static ACL + 3 reporting methods; some contain many subcases.
Corporate ACL matrix	5,334 cases checked.	254 Corporate source host values x 3 protected targets x 7 protocol/port cases.
Admin source restriction	253 cases checked.	Admin hosts .2 through .254; only .10 allowed for modeled SCADA RDP.
Reporting validation	13 / 13 checks passed.	5 policy cases + SELECT + 6 rejected operations + unchanged data.
Browser interaction tests	Passed.	Presets, custom denial, invalid input, reply buttons, audit, export, reset and responsive layout.
Local hosting tests	Passed in live and static modes.	Correct reporting-mode labels, 20/20 and 13/13 suites, downloads, health, malformed JSON-object handling and path escape denial.

Suite	Verified result	Meaning
Current public Render URL	Not independently verified from preparation workspace.	Outbound policy blocked verification; test the public site before class.

Do not add these numbers into one test count. Reporting checks are also exercised by Python test methods; static matrices are subcases, and browser/Python models have overlapping policy coverage. Passing these finite cases does not prove universal security correctness or 100% threat prevention.

16. Mapping to the original Word report

Original objective	Delivered status	Remaining boundary
1. Four-zone architecture	Address zones and policy design implemented; Cisco build kit supplied.	Actual VLAN/trunk forwarding not run.
2. NAT/PAT	Corporate/Admin simulated PAT implemented and tested.	Report objective says all zones, but annexure excludes OT; implementation follows OT exclusion.
3. Default-deny ACLs	Model default deny and logs; Cisco text rules.	No actual IOS ACL counters or syslog execution evidence.
4. HTTPS/DNS/Telnet	Protocol/port decision checks implemented.	No modeled-flow TLS handshake; same-VLAN Telnet bypass limitation.
5. IT/OT isolation and reporting	Ordinary Corporate-to-OT deny; jump-host rules; separate SQLite/read-only and exact-tuple demo.	Reporting exception is not deployed across real network zones.
6. Stateful analysis	Tuple reply matching and stateless ACK/RST comparison explained/tested.	No complete TCP state machine or deployed IOS zone-based firewall.
7. Empirical validation	Model, SQLite and browser evidence generated.	Original Cisco packet validation remains unrun.

Original TC-01 through TC-05 require NAT table evidence, corporate HTTPS success, IT-to-OT ACL drop, authenticated jump-host SSH, and blocked OT Internet packets. The project models those policy outcomes but has not reproduced genuine Packet Tracer traffic, authenticated SSH, or IOS counters. A teacher must accept the no-install alternative if Packet Tracer is mandatory.

17. Hosting and deployment lifecycle

• Render imports the GitHub repository and uses render.yaml. Its build runs python3 tools/build_site.py and python3 -m unittest discover -s tests -v.

• Render starts python3 server.py --host 0.0.0.0. The hosting platform supplies PORT; /health is the configured health-check path.

• At startup a new temporary local SQLite fixture is initialized. /api/reporting-validation runs real read-only checks inside the service.

• The dashboard remains a browser policy model even when served over public HTTPS. Render's browser-to-site TLS does not prove a simulated Corporate-to-Internet flow established TLS.

• Vercel serves the checked-in site directory with vercel.json. The reporting UI labels results as recorded evidence because this configuration has no live backend.

• Repository pushes can trigger Render auto-deployment when the integration is enabled. A Git push is not itself proof the new version is live.

The manifest requests a free Render plan. Current eligibility, sleeping behavior and limits depend on Render account/platform policy. An idle service may need time to wake. No local installation is needed to show the hosted project. Avoid claiming always-on availability or persistent industrial data storage.

18. Local reproduction and development

```
cd firewall
python3 -m unittest discover -s tests -v
python3 reporting_demo.py
python3 tools/build_site.py
python3 server.py

# Hosting-style bind, only where appropriate:
python3 server.py --host 0.0.0.0 --port 8000
```

Open the local loopback address shown by the server on your own computer; use the hosted URL for the teacher. The dashboard can also open as a standalone HTML file, but live SQLite reporting requires the Python service. A standalone local reporting script can generate its own result page using Python's built-in SQLite.

To reproduce browser automation, Playwright and a compatible Chromium must already be available, and the expected server ports must be running. They are not required just to view the app. The document generator requires its development-only libraries; normal Render builds copy already-generated documents instead of regenerating them.

19. Five-to-ten-minute classroom demonstration

Time	Action	What to say
0:00-1:00	Open dashboard and show four zones.	This is a no-install policy simulation for a manufacturing network.
1:00-2:00	Explain threat model and default deny.	A compromised corporate device should not initiate arbitrary OT connections.
2:00-3:00	Run Validation & evidence; show 20/20 and PAT table.	Two corporate clients share one public address with distinct simulated ports.
3:00-4:00	Select IT to SCADA; evaluate; show DENY.	The modeled gateway blocks lateral traffic and records the reason.
4:00-5:00	Show jump-host SCADA RDP; change source to .11.	The authorized host is permitted; the other admin host is denied.
5:00-6:00	Show matching and wrong-source replies.	Tuple matching demonstrates the idea of tracking, but not full TCP SPI.
6:00-7:30	Open reporting page; refresh; show 13/13 and telemetry.	These are real SQLite reads/rejected writes, with synthetic data; network tuple is separate.
7:30-9:00	Show GitHub, report and limitations.	Cisco configurations are supplied, but no Packet Tracer execution is claimed.
9:00-10:00	Answer questions.	State exactly which behavior was implemented and which was only modeled.

Before class, check /health, open the reporting page and confirm its mode, and run the dashboard suite in the same browser you will use. Keep PDFs, JSON evidence and local files as a fallback if Wi-Fi or Render startup is slow. Do not label a failed or unavailable live check as passed.

20. Common teacher questions and accurate answers

Question	Answer
Is this a real firewall?	No. It is a network-policy simulator plus a real local read-only SQLite service, with optional Cisco configuration text.
Why four zones?	They separate different trust levels and give IT/OT/admin traffic an explicit boundary.
Why default deny?	Unknown or unauthorized cross-zone flows are rejected unless a specific permit applies.
Why are VLANs needed?	In the conceptual network they create separate Layer 2 segments; router/firewall rules govern traffic crossing those segments.
Does PAT provide encryption?	No. PAT translates addresses/ports. TLS or SSH would provide encryption in real connections.
Why not allow all Admin hosts?	A dedicated jump host reduces the number of authorized initiators; the modeled permissions are intentionally narrow.
Does established mean stateful?	Not for a Cisco stateless ACL. It checks ACK/RST flags, not a real connection table.
Can ACLs make SQL read-only?	No. ACLs restrict endpoints and ports; database/application permissions restrict reads versus writes.
Are the PLC records real?	No. They are synthetic SQLite rows used for reproducible verification.
Why TCP 8443?	It is an explicit proposed HTTPS reporting-service design assumption; no such industrial listener is deployed here.
Why is the reporting API public?	It is a classroom demo returning synthetic data; production would need authentication, authorization and hardening.
Is the reporting tuple protecting that API?	No. Source/destination tuple checks are a separate simulated policy, not HTTP client authentication.
What happens on restart?	The synthetic database is recreated and in-memory API state resets. Browser state resets on reload/reset.
What stops same-VLAN Telnet?	This gateway design does not. Endpoint controls, port isolation or further segmentation are needed.
Did Packet Tracer run?	No. The original tool-specific execution requirement remains unmet; present this as an alternative implementation.
What would production require?	A supported firewall/router, verified real segmentation, strong identity/TLS, endpoint controls, monitoring and industrial-aware inspection.
What did you actually test?	20 browser checks, 16 Python methods, 13 reporting checks and local live/static deployment paths; scopes overlap and stay separate.

21. Troubleshooting during presentation

Symptom	Check / response
Site loads slowly	Allow an idle Render instance time to wake, then check /health. Use saved local files if unavailable.
Reporting page says recorded evidence	You may be on static hosting or the live API was unreachable. Do not claim those displayed checks just ran live.
Browser counters differ from Python /api/state	Expected: the two policy models have separate state.
PAT table is empty	Evaluate a permitted cross-zone Internet flow. Denied and internal flows should not receive Internet PAT.
Matching reply button cannot run	First evaluate a permitted cross-zone flow; same-VLAN or denied flows do not create that modeled session.

Symptom	Check / response
Generic Packet Tracer server has no RDP/SSH listener	A permit rule does not create a service; actual application tests are unsupported unless a listener exists.
A check fails	Keep the failed result, inspect the input/matched rule/service response and report its actual status.
Public page does not show the latest guide	Check Render deployment status after the corresponding GitHub commit; repository updates and running versions are separate.

22. Limitations and next engineering steps

• No Cisco Packet Tracer or real IOS execution; no genuine .pkt topology, runtime ACL counters or Cisco packet captures.

• No actual Corporate-to-OT VLAN boundary, SCADA/PLC process, industrial database sync, or modeled HTTPS/SSH/RDP handshake.

• No full TCP state, idle expiration, protocol-specific ICMP/NAT engine, production PAT resource management, or intrusion detection.

• HTTPS can carry permitted malicious/exfiltration traffic; same-VLAN paths and physical USB threats remain.

• The public HTTPServer is suitable for a small educational demo, not a hardened high-concurrency security service.

• For production: industrial NGFW/DPI, authenticated reporting gateway, least-privilege DB accounts, endpoint hardening, centralized monitoring, backups and tested recovery.

• For the original Cisco assignment: use Packet Tracer/IOS, collect the five required network tests and resolve unsupported application services honestly.

23. Key terminology

Term	Meaning
IT / OT	Information Technology / Operational Technology.
SCADA / PLC	Supervisory Control and Data Acquisition / Programmable Logic Controller.
VLAN	Virtual LAN: a logical Layer 2 segment.
ACL	Ordered Access Control List rules matching addresses, protocols and ports.
Layer 3 / Layer 4	Network layer addressing / transport-layer protocol and port information.
Default deny	Reject unmatched cross-zone traffic instead of allowing it by default.
NAT / PAT	Network Address Translation / Port Address Translation.
Jump host	Restricted system used to initiate administrative access.
Stateful inspection	Tracks real connection/protocol state; stronger than isolated header/flag checks.
Read-only access	Connection permissions allow reading but reject modification operations.
API	Application interface used by the reporting page and policy clients.
Fixture	Known synthetic test data initialized for repeatable checks.
Data diode	Hardware-enforced one-way communication device; a future recommendation here.
Air gap	Physical isolation; the routed conceptual network is not an air gap.

24. References and final presentation statement

• Design reference: uploaded NS-P02_Report (2).docx; its existing claimed passes are not reproduced evidence for this version.

• Source repository: https://github.com/JaiKukreja-7/firewall

• Hosted URL supplied by owner: https://ns-p02-firewall-lab.onrender.com

• Repository documentation: README.md; docs/report-audit.md; docs/reporting-exception.md; docs/packet-tracer.md; docs/deployment.md.

• Evidence: deliverables/validation-evidence.json; deliverables/reporting-evidence.json; tests/ and browser automation scripts.

Suggested closing statement: "This project demonstrates manufacturing network policy through a four-zone browser simulation, source-port-based PAT examples and restricted management flows. It also demonstrates real local SQLite read-only enforcement. I have verified the stated model and database checks and supplied Cisco configuration references, but I do not claim actual Packet Tracer execution, real industrial-network segmentation or production security readiness."